AXON-RFC-006 Appendix B: easynet.web_app

Technical design, build pipeline, Hub routing, `actions.*` namespace, and implementation plan for full-stack agent-deployed applications

Silan Hu
National University of Singapore

Draft v0.3, April 28, 2026

Abstract

This appendix specifies `easynet.web_app`, the state_type that supports a full modern web deployment: a real React/Vue/Next-style frontend project authored by an agent (typically Claude Code) inside a project directory under `~/.easynet/pages-project/<slug>/`, built into a static `dist/` bundle, and served alongside a backend whose endpoints are not pretend HTTP handlers but genuine EasyNet abilities. The Hub plays the role of a state-aware nginx: it serves the frontend bundle, translates `POST /api/<verb>` requests into ability invocations on the owner agent, proxies WebSocket subscriptions to ability streams, and exposes a structured `agent_view` on `/__easynet/agent`. Beyond serve-and-translate, this appendix introduces the `actions.*` ability namespace — queryable endpoints (`actions.list_for_page`, `actions.get`, `actions.search`) through which a peer agent obtains EAL fragments and missions *relevant to a specific page or intent*, on demand, without bulk-downloading a static manifest. The agent-skill seeder (`easynet-pages-author`) installs in every agent's workspace at first daemon contact so that authoring a web_app is a learnable skill, not undocumented protocol knowledge. The appendix covers: project location and lifecycle; the state machine (canonical + runtime overlay); the long-running build pipeline; the Hub routing matrix; the `actions.*` contract and the `/easynet-for-agents/<verb>` HTTP mirror; agent_view's role in this redesign; the five PR slices required to land it in `EasyNet-Cli`; and an end-to-end worked example of an agent authoring a Vite + React shop, declaring its API and actions surface, building, publishing, serving, and being audited.

Contents

1 Position and lineage **2**
1.1 Why this is not nginx . . . 3
1.2 What changed from v0.1 . . . 3

2 Goals and non-goals **3**
2.1 Goals . . . 4
2.2 Non-goals . . . 4

3 Project model **4**
3.1 Project location . . . 5
3.2 easynet.app.toml . . . 5
3.3 Why declared API and actions surfaces, not auto-detected . . . 6
3.4 Agent skill seeded by default . . . 7

4 State machine **7**
4.1 state_type and state_key . . . 7
4.2 Canonical fields . . . 8
4.3 Canonical state space . . . 8
4.4 Runtime overlay state space . . . 9

4.5 Transition list . . . 9

5 Build pipeline **9**
5.1 Steps of webapp.build@v1 . . . 9
5.2 TreeBlob format . . . 10
5.3 Build executor . . . 10

6 The actions.* ability namespace **10**
6.1 Why a namespace, not a static directory . . . 11
6.2 Recommended verbs . . . 11
6.3 Action descriptors . . . 11
6.4 What EAL actually is (and what it isn't) . . . 12
6.5 Example: actions.list_for_page + actions.get . . . 12
6.6 Hub mirror semantics . . . 13
6.7 What the protocol does NOT enforce . . . 14

7 Hub routing model **14**
7.1 Hub behavior matrix per state . . . 14
7.2 Action dispatch . . . 15

8 agent_view **15**

9 Architecture **16**
9.1 Module map (EasyNet-Cli) . . . 16
9.2 Daemon wiring . . . 17
9.3 Hub wiring . . . 17

10 Implementation plan **18**
10.1 P-B1: build executor + TreeBlob . . . 18
10.2 P-B2: webapp.* abilities . . . 18
10.3 P-B3: Hub static + API + actions routing . . . 18
10.4 P-B4: WebSocket proxy + SPA fallback . . . 19
10.5 P-B5: agent_view + meta endpoints + skill seed . . . 19

11 End-to-end worked example **19**
11.1 Setup . . . 19
11.2 Author the project . . . 19
11.3 Build and serve . . . 20
11.4 Browser fetch . . . 20
11.5 API call from the browser . . . 20
11.6 Peer agent discovers the actions . . . 20
11.7 Audit replay . . . 21

12 Open questions deferred to RFC-006-B v2 **22**
12.1 Coverage check . . . 22

1 Position and lineage

[derived] The web evolved from a static-document medium into a full application platform; this appendix tracks the same arc as the second of two appendices in RFC-006:

Appendix	What it models
A (`easynet.page`)	A single document. Author writes HTML / Markdown; Hub serves it. The minimal reference: one canonical state, no runtime overlay, no build step, no API surface.
B (`easynet.web_app`)	A full application. Author writes a real frontend project, a build pipeline produces a static bundle, the application's API endpoints ARE EasyNet abilities, and a peer agent can query the owner for "what can I do on this page?" through the `actions.*` ability namespace.

We keep both. Pages remains the simplest verification of the protocol's normative core; web_app is the first state type that exercises long-running canonical transitions, runtime overlay, full Hub routing, and agent-actionable surface discovery.

1.1 Why this is not nginx

Nginx serves a static directory and reverse-proxies to a separately-deployed backend. The backend is a black box — nginx neither understands its state, nor knows what endpoints it offers, nor can answer "what can a caller do on this page" to a peer asking through structured discovery. The Hub specified here serves the same kinds of bytes nginx would, but its routing decisions are derived from the state object's canonical state, its API endpoints are abilities introspectable through the same registry that backs every other EasyNet caller, and its discovery surface (`actions.*`) is not a side-band manifest but a first-class queryable namespace owned by the owner agent. Nginx is HTTP plumbing; the Hub is a state-projection layer that happens to speak HTTP at its outer boundary, with an EAL-aware dual on the inside.

1.2 What changed from v0.1

Three structural changes since the previous draft:

1. **Project home.** Project directory moved from `<workspace>/web_apps/<slug>/` to a dedicated, global `~/.easynet/pages-project/<slug>/`. Rationale: the project belongs to a state object whose owner is already encoded in the state_key, not to the agent's home directory.

2. **Agent skill seeded by default.** A bundled skill, `easynet-pages-author`, is installed into every agent's workspace at first daemon contact, mirroring the way `easynet-ability-crud` is already seeded. The skill teaches an agent how to scaffold, declare, build, serve, and discover web_apps. Authoring is a learnable skill, not folklore.

3. **actions as a queryable ability namespace.** The previous draft proposed an "actions_view" that was a static directory of EAL files served alongside the page. That design forced peer agents to bulk-download every action whether or not relevant to their current page or intent — which is the opposite of how a peer would want to consume agent-actionable surface. The new design introduces `actions.*` as an EasyNet ability namespace: a peer agent calls `<owner>.actions.list_for_page(page_path)` and receives only the actions relevant to that page, on demand. The HTTP mirror at `/easynet-for-agents/<verb>` exists for browser-side use but the protocol-level contract is the ability, not the URL.

2 Goals and non-goals

2.1 Goals

[normative] An implementation conforming to this appendix:

1. Lets an agent author a real frontend project under `~/.easynet/pages-project/<slug>/` using whatever toolchain the agent prefers (Vite, Next.js, Remix, …); EasyNet does not invent a project structure.
2. Provides a long-running canonical transition, `webapp.build`, that runs the project's declared build command, captures the resulting `dist/` as an immutable content-addressed bundle, and advances canonical state on success or failure.
3. Provides runtime overlay transitions (`webapp.serve`, `webapp.stop`) that bind a Hub mapping; canonical state is unchanged.
4. Translates HTTP requests on the Hub along three orthogonal channels: static-bundle fetch, API ability invocation (`/api/<verb>`), and WebSocket ability subscription (`/ws/<topic>`).
5. Exposes `agent_view` on `/__easynet/agent` as a typed JSON document of the application's state and discoverability metadata, including a pointer to the `actions.*` namespace.
6. Defines the `actions.*` ability namespace (§6): a queryable, owner-implemented surface through which peer agents request EAL fragments and missions on demand. The HTTP mirror `/easynet-for-agents/<verb>` is an alias for ability invocation, not a static directory.
7. Seeds the `easynet-pages-author` skill into each agent's workspace so authoring a web_app is a documented capability of the agent itself (§3.4).
8. Is implementable in five PR slices (§10), each independently reviewable.

2.2 Non-goals

[derived] Out of scope for this appendix:

- TLS termination, HTTP/2 negotiation, multi-instance Hub clusters — operations work, not protocol work.
- Choice of frontend framework — the appendix uses React/Vite as the worked example because Claude Code defaults to it; the protocol is framework-agnostic.
- Server-side rendering with persistent process — Phase 1 is static + Hub-routed API + WebSocket; SSR requiring a Node.js process per request is RFC-006-B v2.
- Cross-realm or cross-node application federation — Phase 1 is single owner / single node.
- Authenticated humans (`principal/human-auth`) — Phase 1 is `principal/human-anon` only.
- "Three-view equivalence" as a strict protocol invariant — enforcing that `human_view` and actions content carry exactly the same information is desirable but not protocol-encodable in v1; the `actions.*` namespace is the owner's voluntary statement of agent-actionable surface, not a derivation.

3 Project model

3.1 Project location

[normative] The state object's source-of-truth on disk is a project directory at:

```
~/.easynet/pages-project/<slug>/
  easynet.app.toml           # the EasyNet-side declaration
  package.json               # owned by the agent's toolchain
  package-lock.json          # ditto
  src/                       # framework-shaped source
  public/
  vite.config.ts             # or next.config.js, etc.
  (optionally) anything else the framework needs
```

EasyNet **owns** only `easynet.app.toml`; the rest is the framework's. The location is global (one directory tree per host, slug-keyed) rather than per-agent (`<workspace>/...`) because the state_key already carries owner identity — nesting the directory under the agent's workspace would encode ownership twice and complicate Hub access (the Hub reads artifacts via the blob store, but a debug operator inspecting a project should not have to know which agent owns a given slug).

The directory is created by the daemon on `webapp.create`; the agent populates it with framework files via its own toolchain (`npm create vite`, etc.). The daemon does not write into this directory after creation except through transitions on the state object's behalf (e.g. clearing on `webapp.delete`). The blob store at `~/.easynet/blobs/` is the canonical source of bytes served by the Hub; the project directory is the agent's working copy.

Invariant — WB-INV-1

After a successful `webapp.build`, the Hub serves bytes exclusively from the blob store keyed by `build_artifact_hash`. The project directory's `dist/` may exist on disk for the agent's debugging convenience but is **never** read by the Hub. Reading the project's `dist/` for serving purposes is non-conformant.

3.2 easynet.app.toml

[normative] The single file EasyNet reads to understand a web_app:

```
# easynet.app.toml — the only file EasyNet authors and reads.
# The rest of the project belongs to the framework.

[manifest]
name         = "shop"
description  = "Tiny ceramic shop, agent-built."
visibility   = "public"            # private | realm | public

[build]
command      = "npm install && npm run build"
dist_dir     = "dist"              # relative to project root
build_timeout_secs = 600
node_version = "20"                # advisory; executor may pin

[serve]
mode         = "static-only"
               # "static-only" : no backend process; API is satisfied
               #                 by ability invocations from the Hub.
               #                 The default and the Phase-1
               #                 production shape.
```

```
                # "process"      : a backend process is started;
                #                  reserved for v2 (SSR support).

[api]
# Abilities the FRONTEND is allowed to call through the Hub's
# /api/<verb> route. Each entry is a fully-qualified ability name
# the owner agent has registered. The Hub will refuse to translate
# a /api/<verb> request whose verb is not on this list.
abilities = [
  "shop.list_items",
  "shop.checkout",
  "shop.order_status",
]

[actions]
# Owner-implemented `actions.*` abilities exposed via the
# /easynet-for-agents/<verb> mirror. Each entry is a verb the
# owner agent has registered under the actions.* namespace.
# Peer agents call these to discover EAL fragments and missions
# on demand. See §6 for the protocol.
abilities = [
  "actions.list_for_page",
  "actions.get",
  "actions.search",
]

[routes]
# nginx-shaped routing rules. Order is significant.
rules = [
  { match = "exact:/",                      action = "static:dist/index.html" },
  { match = "prefix:/assets/",              action = "static:dist/assets/" },
  { match = "regex:^/api/([a-z][a-z0-9_]*)$",
    action = "api:$1",
    methods = ["GET","POST"] },
  { match = "regex:^/easynet-for-agents/([a-z][a-z0-9_]*)$",
    action = "actions:$1",
    methods = ["GET","POST"] },
  { match = "prefix:/ws/",                  action = "ws:$path" },
  { match = "exact:/__easynet/agent",       action = "agent_view" },
  { match = "exact:/__easynet/health",      action = "health" },
  { match = "exact:/__easynet/receipts",    action = "receipts" },
  { match = "fallback",                     action = "spa:dist/index.html" },
]

[csp]
extra_script_src  = []
extra_style_src   = ["'unsafe-inline'"]
extra_connect_src = ["self"]
```

A minimal app gets by with three sections (`[manifest]`, `[build]`, `[api]`); `[actions]` is optional (§6 explains why) and routes default to the standard SPA + actions layout shown above.

3.3 Why declared API and actions surfaces, not auto-detected

[normative] Both the `[api]` and `[actions]` surfaces are declared, not discovered. The Hub refuses to translate `/api/<verb>` or `/easynet-for-agents/<verb>` when the corresponding verb is not declared.

Invariant — WB-INV-2

The Hub **MUST** refuse to translate `/api/<verb>` unless `<verb>` appears in the web_app's currently-Built canonical state's `api_surface_hash` (the canonicalised `[api].abilities` list). The same rule applies to `/easynet-for-agents/<verb>` relative to `actions_surface_hash`. The refusal MUST be HTTP 404 (not 403) to avoid confirming the existence of internal abilities — the application-layer instance of the trust-boundary discipline TR-INV-12 establishes at the identity layer.

3.4 Agent skill seeded by default

[normative] The daemon, on the first contact with each agent, seeds a bundled skill named `easynet-pages-author` into that agent's workspace at the standard skill location:

```
<workspace>/skills/easynet-pages-author/SKILL.md
<workspace>/.agents/skills/easynet-pages-author/SKILL.md
```

The skill is identical in shape to the existing `easynet-ability-crud` skill (commit 3bc6c70). It documents:

- how to scaffold a frontend project (`npm create vite@latest <slug> -- --template react-ts`);
- the standard project location (`~/.easynet/pages-project/<slug>/`) — to be used *after* `webapp.create` mints the state_key and creates the directory; the agent does `cd` into the directory the daemon prepares;
- the structure of `easynet.app.toml`, including the `[api]` and `[actions]` sections;
- how to register `api.*` abilities through the existing `easynet-ability-crud` skill;
- how to implement the `actions.*` namespace (§6) so that peer agents can discover the application's actionable surface;
- the lifecycle: `webapp.create` → populate project → `webapp.build` → `webapp.serve`;
- how to invoke `webapp.update_config` for routing or CSP changes that don't require a rebuild;
- troubleshooting (build failures, port conflicts, visibility-pinned 404s).

The skill is a Phase-1 deliverable in P-B5 (see §10). It is not part of canonical state — it lives in the agent's workspace, owned by the agent, and is re-seeded only when the daemon detects a missing or out-of-date skill at boot.

4 State machine

4.1 state_type and state_key

[normative]

```
state_type:  easynet.web_app

state_key:
    (realm: string, owner_agent_uri: string, app_slug: string)

URA form:
```

```
easynet:///r/<realm>/agent/<owner>/web_app/<slug>
```

`app_slug` matches the slug ABNF defined in Appendix A's pages section. The directory at `~/.easynet/pages-project/<app_slug>/` is created on `webapp.create` and removed on `webapp.delete`.

4.2 Canonical fields

[normative] Per CSH-INV-2 (main §2.5) every large value is held by content hash:

```
canonical_fields(web_app) = {
    manifest_hash:        bytes(32),  // [manifest] section
    build_config_hash:    bytes(32),  // [build] section
    eal_hash:             bytes(32),  // SHA-256 of the web_app's
                                      // EAL source (markdown
                                      // documentation in `// ...`
                                      // comments + mission decls).
                                      // Per Appendix A PA-INV-2,
                                      // EAL is the canonical
                                      // agent-facing artifact;
                                      // api_surface_hash is derived
                                      // from EAL at update_config
                                      // time, not separately authored.
    api_surface_hash:     bytes(32),  // [api].abilities (sorted);
                                      // recomputed from EAL by the
                                      // daemon, not written by hand
    actions_surface_hash: bytes(32),  // [actions].abilities (sorted)
    routes_manifest_hash: bytes(32),  // [routes].rules (in order)
    csp_hash:             bytes(32),  // [csp] section
    source_snapshot_hash: bytes(32),  // last successful build's
                                      // source tree hash (null
                                      // before first build)
    build_artifact_hash:  bytes(32),  // dist tree hash (null
                                      // until first Built)
    state_value:          string,
    visibility:           string,
    version:              uint64,
}
```

Every field is tagged `canonical:true` in the schema; schema-driven projection π produces the projection over exactly these keys.

4.3 Canonical state space

state_value	Meaning
`Created`	easynet.app.toml registered; project directory exists; no build attempted yet.
`Building`	A `webapp.build` is in flight. Transient. Visible only via OperationalReceipts; never the terminal canonical state.
`Built`	At least one successful build exists. `build_artifact_hash` non-null. May be Served.
`Failed`	The most recent build failed. Owner can iterate and retry.
`Deleted`	Terminal; project directory and blob references cleared.

4.4 Runtime overlay state space

runtime	Meaning
`Stopped`	No serving overlay associated with this app.
`Starting`	`webapp.serve` invoked; Hub is binding the slug into its router.
`Running`	Hub is actively translating `/<slug>/...` requests for this app.
`Crashed`	The Hub lost its mapping (process restart, config-reload error, etc.); canonical state untouched. Recoverable via `webapp.serve`.

4.5 Transition list

Transition	Class	Notes
`webapp.create@v1`	canonical	$\emptyset \rightarrow$ Created. Mints state_key, creates project directory, persists hashes of declared sections.
`webapp.update_config@v1`	canonical	Created/Built/Failed $\rightarrow$ same. Replaces declared config (manifest, build, api, actions, routes, csp). Does NOT trigger a build.
`webapp.build@v1`	canonical	Created/Built/Failed $\rightarrow$ Building (transient) $\rightarrow$ Built|Failed. Long-running. See §5.
`webapp.delete@v1`	canonical	* $\rightarrow$ Deleted. Implicit `webapp.stop` as side effect; clears project directory.
`webapp.serve@v1`	operational	Stopped/Crashed $\rightarrow$ Starting $\rightarrow$ Running. Requires canonical = Built.
`webapp.stop@v1`	operational	Starting/Running/Crashed $\rightarrow$ Stopped.
`webapp.get@v1`	query	Returns canonical + runtime snapshot.
`webapp.get_eal@v1`	query	Mandatory. Returns the UTF-8 EAL source whose SHA-256 equals `eal_hash`. HTTP mirror: `GET /__easynet/eal`. Mirrors Appendix A PA-INV-2.
`webapp.list@v1`	query	Lists web_apps owned by an agent.

5 Build pipeline

5.1 Steps of webapp.build@v1

[normative] On `webapp.build` the executor:

1. Reads `~/.easynet/pages-project/<slug>/easynet.app.toml` and validates it.
2. Computes `source_snapshot_hash`: a Merkle hash over the project directory excluding `node_modules/`, `dist/`, `.git/`, and any path the framework's `.gitignore` excludes.
3. Emits `build.started` OperationalReceipt with the source-snapshot hash, declared command, expected timeout.
4. Spawns the build command in the project directory. Stdout / stderr are tee'd; periodic `build.progress` OperationalReceipts are emitted (lossy).
5. On exit:

- Status 0 + `dist_dir` non-empty: walk the `dist_dir` tree; compute `build_artifact_hash` as a Merkle hash; write the entire tree to the blob store as a TreeBlob (a JCS-canonical manifest of file paths and individual blob hashes; the file blobs are stored as ordinary content-addressed blobs).
- Status non-zero or empty `dist_dir`: prepare a failure-reason payload and store its hash.

6. Emit one CanonicalReceipt: `build.completed` (success) or `build.failed` (failure). The sole canonical advance for the transition; `post_state_hash` reflects Built or Failed; `state_version` +1.

The transient `Building` state value, the streaming progress, and the build-tool stdout are all observability; they never enter the canonical hash.

5.2 TreeBlob format

[normative]

```
TreeBlob (JCS-canonical JSON)
{
  "version": 1,
  "entries": [
    { "path": "index.html",
      "size": 2148,
      "blob": "<sha256-of-file-bytes>",
      "mode": "0644" },
    { "path": "assets/index-DfX9k.js",
      "size": 187321,
      "blob": "<sha256>",
      "mode": "0644" },
    ...
  ]
}
```

The tree's hash is `sha256(jcs(TreeBlob))` — `build_artifact_hash`. Each individual file is also a content-addressed blob.

5.3 Build executor

[normative] Phase 1 ships a local build executor: the build runs in a subprocess of the EasyNet daemon owning the agent. The owner signs the resulting CanonicalReceipt directly. Delegated build (source on agent A, build on agent B) is the future: the protocol shape is already in place (main §5.3) but the executor is not.

Open decision

Open: build sandboxing. The build subprocess inherits daemon process privileges. Phase 1 mitigation: build runs with a restricted `PATH`, no inherited env vars except `NODE_VERSION` and `HOME`, stricter `ulimit`. Full sandboxing (seccomp / pledge / Docker) is RFC-006-B v2.

6 The actions.* ability namespace

6.1 Why a namespace, not a static directory

[derived] A peer agent visiting a web_app does not want to download every EAL fragment the owner ever wrote; it wants *the actions relevant to the page or intent it has now*. Static directories of EAL files force bulk download and client-side filtering — exactly the inefficiency the previous draft had. The redesign treats the agent-actionable surface as a queryable resource.

The owner agent registers abilities under the `actions.*` namespace. A peer agent — or the Hub on behalf of a browser — invokes those abilities by name. The owner's implementation decides what to return: the page-relevant set, an intent-matched set, the full directory, a single named action, or any future form. Because actions are abilities, every existing protocol mechanism applies: `ability_surface`, schema, scope check, receipts, audit, federation routing.

6.2 Recommended verbs

[normative] Owners are encouraged but not required to implement a standard set:

Ability	Purpose
`actions.list_for_page(page_path)`	Return the actions relevant to `page_path`. The most common entry point: an agent that just landed on a page asks "what can I do here."
`actions.get(name)`	Return the full definition of one named action — typically the EAL or mission text plus a structured `inputs` schema.
`actions.search(intent)`	Return a ranked list of action descriptors matching a free-text intent string. Owner-implementation-defined ranking.
`actions.list_all()`	Return the full directory. Equivalent to a sitemap; useful for crawlers and for mirroring.

The verb set is a *convention*, not protocol. An owner may implement a subset, a superset, or none; an agent invoking a non-existent verb gets a normal `MethodNotFound` error. The Hub's `/easynet-for-agents/<verb>` mirror simply forwards to whatever verb the owner registered.

6.3 Action descriptors

[normative] A descriptor returned by any `actions.*` verb has the shape:

```
{
  "name":     "checkout",          // owner-namespaced
  "kind":     "eal" | "mission" | "form_map",
  "summary":  "Place a single order.",
  "url":      "/easynet-for-agents/get?name=checkout",
                                   // dereference (POST with params)
                                   // to receive a specialised mission
  "params_schema": {               // schema for the params object
                                   //   passed to actions.get;
                                   //   owner inlines them into the
                                   //   returned mission as literals
    "item_id": { "type":"string", "required":true,
                 "doc":"see actions.search to find IDs" },
    "qty":     { "type":"uint",   "default":1 }
  },
  "result_render": {               // optional, for form_map
```

```
    "success_template": "Order {order_id}, ETA {eta}",
    "error_template":   "Sorry, {error_message}"
  }
}
```

The body of an action — an EAL fragment, a mission, or a form_map JSON — is whatever `actions.get` returns. Because the body is a value returned by an ability, not a file on disk, the owner may compute it dynamically (e.g. inject the current user's session into a mission) without violating any protocol rule. View determinism (TR-INV-6) does not apply because actions are abilities, not views.

6.4 What EAL actually is (and what it isn't)

[normative] The mission bodies that `actions.get` returns are EAL programs in the syntax this codebase already implements (`src/eal/`). EAL is a deliberately-narrow language:

- A mission is a *static DAG* of `call` steps. No `if`, no `fail`, no expression evaluation, no `return`. Every step's args are filled in when the mission text is written.
- Argument syntax is `with { key = value, …}` (`=`, not `:`). Values are string / int / float / bool literals, or `prev_step.output` variable references.
- Bindings: `let x = call …`. Variables can be referenced only as `x.output`.
- Step options are exactly `timeout N`, `retries N`, `on_failure {abort | skip | retry | continue}`, `optional`.
- The only block form is `loop` with `body {…} verify {…}`.

This is the actual grammar in `src/eal/lexer.rs` and `src/eal/ast.rs`. There is no `inputs` clause, no `$variable` interpolation, no in-mission control flow beyond the `loop` block.

Implication for actions. Because mission text is static, an action that needs caller-supplied parameters cannot "accept inputs" in the body. Instead the protocol uses **owner-side specialisation**: the peer agent passes the parameters it wants to `actions.get(name, params)`; the owner agent inlines those parameters as literals into the mission text it returns. The peer then runs the resulting, fully-literal mission verbatim through `mission run`.

This preserves the receipt chain. Each `call` in the mission is a real EasyNet invocation that goes through the existing dispatcher and produces a CanonicalReceipt; the mission's first step cites `actions.get`'s receipt as its `causal_context.Scalar` so a later auditor can prove "this mission run originated from a specific `actions.get` call to alice."

6.5 Example: actions.list_for_page + actions.get

```
$ easynet ability invoke alice.actions.list_for_page \
    --args '{"page_path":"/cart"}'

[
  { "name":"checkout", "kind":"mission",
    "summary":"Buy the items currently in this user's cart.",
    "url":"/easynet-for-agents/get?name=checkout",
    "params_schema":{                          // peer passes these to
      "address":{"type":"string","required":true},  // actions.get
      "qty":    {"type":"uint",  "default":1}
    } },
  { "name":"clear_cart", "kind":"eal",
```

```
    "summary":"Empty the cart.",
    "url":"/easynet-for-agents/get?name=clear_cart" }
]

# Peer asks owner to specialise the checkout mission for its
# concrete inputs. Owner inlines them as literals into a real EAL
# mission (no $variable, no inputs block).
$ easynet ability invoke alice.actions.get \
    --args '{"name":"checkout",
             "params":{"address":"123 Main St","qty":1}}'

mission "checkout" {
  let cart = call "shop.cart_contents" on "alice" with {} timeout 5

  let order = call "shop.checkout" on "alice" with {
    items   = cart.output,
    address = "123 Main St",
    qty     = 1
  } timeout 30
}
```

The peer agent now runs the returned text directly:

```
$ easynet mission run /tmp/checkout.eal --cite-receipt <actions.get-receipt-id>
```

The `--cite-receipt` flag writes `causal_context.Scalar(`*`prior_invocation_id`*`)` into the first step's invocation envelope, so the receipt log records the chain `actions.get` → `shop.cart_contents` → `shop.checkout`. A peer that fails to cite still gets a working run — it just loses the cross-state-object causal link, leaving each step's receipt provable but the "why this mission ran now" answer unrecorded.

Compare with the previous draft's static-directory design: the same agent would fetch `checkout.eal`, `clear_cart.eal`, `list_items.eal`, an index file, then filter — four fetches plus a client-side parameter- substitution pass for one intent, with no causal_context tying the run back to the page that prompted it.

6.6 Hub mirror semantics

[normative] The route action `actions:<verb>` translates an HTTP request

```
GET  /easynet-for-agents/<verb>?<query>
POST /easynet-for-agents/<verb>     {<json>}
```

into an EasyNet ability invocation `<owner>.actions.<verb>` with the query string or JSON body as args, identical to the `api:<verb>` dispatch. The result is returned as JSON (or, when the result is a string that the owner has flagged as `Content-Type: text/eal`, served verbatim with that content type). Any verb not in `actions_surface_hash` is rejected per WB-INV-2.

Note

The HTTP mirror's existence is convenience, not authority. A peer EasyNet agent can equivalently call `<owner>.actions.<verb>` directly through the bus. Both paths produce the same kind of OperationalReceipt and the same caller-context recording. The mirror exists primarily so a browser-side script can build a UI on top of the actions surface (e.g. a "what can I do here" dropdown) without needing an EasyNet client.

6.7 What the protocol does NOT enforce

[derived] Three deliberately-omitted enforcements:

- **No cardinality between human_view and actions.** An owner whose page shows "12 items for sale" is not protocol-required to make `actions.list_for_page("/")` return 12 items. Honesty between views is an *owner discipline*; the protocol provides the mechanism (both views are signed by the same owner; a peer auditing the chain can detect drift) but does not require equivalence as an invariant. v1's enforcement window is too narrow.
- **No standard EAL/mission text format version.** The body of `actions.get` is whatever the owner returns; the EAL grammar lives in its own RFC and evolves independently. Action descriptors carry `kind` so that a consumer can dispatch on body interpretation.
- **No authoritative ranking or relevance.** `actions.list_for_page` and `actions.search` return owner-defined rankings. A peer wanting independent ranking implements its own.

7 Hub routing model

The Hub is bound to a single port (Phase 1: `127.0.0.1:8787` by default). Every incoming HTTP request is processed in seven steps:

1. Mint envelope: `caller = principal/human-anon/<ip-hash>/<sid>`; `caller_context` from headers per main §3.6.
2. Parse path to `(realm, owner, slug, tail)`.
3. Resolve state. If not Built, return per §7.1; if visibility forbids the caller, return 404.
4. Match `tail` against the `routes_manifest` rules in declared order.
5. Dispatch on the matching action: `static` / `api` / `actions` / `ws` / `agent_view` / `health` / `receipts` / `spa` fallback (see §7.2).
6. Build response (CSP, ETag, cache headers, content-type).
7. Emit one OperationalReceipt with caller, caller_context, matched rule, action, response status. Not canonical.

7.1 Hub behavior matrix per state

Canonical	Runtime	Hub response
`Created`	*	503 "not built". `Retry-After: 60`.
`Building`	*	503 + small progress page (SSE of build.progress).
`Built`	`Stopped`	503 "not serving".
`Built`	`Starting`	503 "starting", `Retry-After: 2`.
`Built`	`Running`	Full action dispatch (below).
`Built`	`Crashed`	503 + reason hash; `Retry-After: 5`.
`Failed`	*	503 "last build failed" + reason hash.
`Deleted`	*	404.

7.2 Action dispatch

static:<path> TreeBlob path lookup, blob fetch, extension-based MIME (allow-list), CSP from `csp_hash`, immutable cache for `/assets/*`.

api:<verb> Validate against `api_surface_hash`, parse JSON body or query string as args, invoke `<owner>.<verb>`, return result body or structured error. Refusal: HTTP 404 (WB-INV-2).

actions:<verb> Validate against `actions_surface_hash`, parse JSON body or query string as args, invoke `<owner>.actions.<verb>`, return result body. Content-Type defaults to `application/json`; if the result body declares `Content-Type: text/eal`, the Hub forwards verbatim under that type (so peers fetching an EAL file via the mirror see proper text). Refusal: HTTP 404 (WB-INV-2).

ws:<path> WebSocket upgrade; first text frame is a `{ ability, args }` JSON; bind to ability stream; forward subsequent frames.

agent_view Return JSON specified in §8.

health Return canonical and runtime states.

receipts Return CanonicalReceipt history bounded by `?from=...&to=...`.

spa:<fallback-path> Same as `static:<fallback-path>` but `Cache-Control: no-cache`.

8 agent_view

[normative] `agent_view` is structured *metadata* about the application: state, version, history, the surfaces (api + actions) the application advertises. It is *not* the action library — the actions live in the `actions.*` namespace and are queried via that namespace, not through agent_view.

```
GET /<realm>/<owner>/<slug>/__easynet/agent

200 OK
Content-Type: application/json

{
  "state_type":  "easynet.web_app",
  "state_key":   { "realm":"default","owner":"alice","slug":"shop" },
  "ura":         "easynet:///r/default/agent/alice/web_app/shop",

  "canonical": {
    "state":              "Built",
    "manifest":           { "name":"shop",
                            "description":"Tiny ceramic shop, agent-built.",
                            "visibility":"public" },
    "build_artifact_hash":"0x4d28ee17...",
    "version":            7,
    "etag":               "TSjuFw0pAvjpAQ=="
  },

  "runtime": {
    "state":     "Running",
```

```
    "started_at":"2026-04-28T09:11:02Z"
  },

  "ability_surface": [
    "webapp.update_config@v1",
    "webapp.build@v1",
    "webapp.stop@v1",
    "webapp.delete@v1"
  ],

  "api": {
    "endpoints": [
      { "verb":"shop.list_items",
        "http":"GET  /api/list_items",
        "schema_url":"/__easynet/schema/shop.list_items" },
      { "verb":"shop.checkout",
        "http":"POST /api/checkout",
        "schema_url":"/__easynet/schema/shop.checkout" }
    ]
  },

  "actions": {
    "namespace": "actions",
    "registered_verbs": [
      "actions.list_for_page",
      "actions.get",
      "actions.search"
    ],
    "http_mirror_prefix": "/easynet-for-agents/",
    "discovery_hint":
      "POST /easynet-for-agents/list_for_page {\"page_path\":\"<path>\"}"
  },

  "history": {
    "versions": 7,
    "latest_canonical_receipt_id": "r_c3d4...",
    "receipt_log_ura":
       "easynet:///r/default/agent/alice/web_app/shop/receipts"
  }
}
```

Why agent_view does not embed the action library. Embedding actions would force agent_view to enumerate every EAL fragment the owner ever wrote — the same bulk-download inefficiency that motivated the actions namespace in the first place. agent_view points at the namespace; the peer agent then calls into the namespace for what it actually needs. The two documents are complementary: agent_view answers "what is this application," actions answers "what can I do on it now."

9 Architecture

9.1 Module map (EasyNet-Cli)

```
src/runtime/state/                    [shared with pages, P-A1]

src/runtime/blob_store/               [shared with pages, P-A1]
  tree.rs                             ← NEW: TreeBlob put/get,
```

```
                                        per-path lookup

src/runtime/agents/webapp_ability.rs  ← NEW (P-B2)
                                      register: create / build /
                                      update_config / serve / stop /
                                      delete / get / list

src/runtime/build_executor/           ← NEW (P-B1)
  mod.rs                              long-running canonical
                                      transition driver
  source_snapshot.rs                  Merkle hash of project dir
  spawn.rs                            subprocess + log streaming
  artifact_capture.rs                 dist tree -> TreeBlob

src/runtime/views/webapp/             ← NEW (P-B5)
  agent_view.rs                       JSON projection of §7
  health.rs
  receipts.rs

src/runtime/hub/                      [extends pages Hub from P-A4]
  router.rs                           ← extended for routes_manifest
                                       and the new actions: action
  static_handler.rs                   ← NEW (P-B3): TreeBlob fetch
  api_handler.rs                      ← NEW (P-B3): /api/<verb>
  actions_handler.rs                  ← NEW (P-B3): /easynet-for-agents/<verb>
  ws_handler.rs                       ← NEW (P-B4): /ws/* upgrade

src/runtime/skills/easynet-pages-author/  ← NEW (P-B5)
  SKILL.md                            agent-facing tutorial seeded
                                      into every workspace
```

9.2 Daemon wiring

The daemon at boot, on top of the pages wiring:

1. Opens the build executor's working directory at `~/.easynet/build-cache/`.
2. Registers `webapp_ability` for each agent.
3. For every web_app whose canonical state is `Building` (i.e. a daemon-restart interrupted a build): marks Failed with reason "daemon restart during build."
4. Seeds `easynet-pages-author` skill into each agent's workspace if not already present or if out of date.

9.3 Hub wiring

The Hub binary (`easynet-hub`, from P-A4) gains in P-B3/P-B4 the `static_handler`, `api_handler`, `actions_handler`, and `ws_handler` alongside the page handler. Top-level dispatch:

```
fn handle(req: HttpRequest) -> HttpResponse {
    let envelope = build_envelope(&req);
    let (realm, owner, slug, tail) = parse_path(&req.uri)?;
    let kind = state_store.classify(realm, owner, slug)?;

    match kind {
        Kind::Page    => pages_dispatch(envelope, owner, slug, tail),
        Kind::WebApp  => webapp_dispatch(envelope, owner, slug, tail),
```

```
        Kind::Unknown => http_404(),
    }
}
```

10 Implementation plan

Five PR slices, sequenceable. P-B1 builds on P-A1 (blob store and state store from pages); P-B3/B4 build on P-A4 (basic Hub).

10.1 P-B1: build executor + TreeBlob

New files. `src/runtime/build_executor/` (4 modules), `src/runtime/blob_store/tree.rs`.

Public API.

```
pub trait BuildExecutor: Send + Sync {
    fn execute(
        &self,
        project_dir: &Path,
        config: &BuildConfig,
        progress: ProgressSink,
    ) -> Result<BuildOutcome>;
}
pub fn source_snapshot_hash(project_dir: &Path, ignore: &[Pattern])
    -> Result<[u8; 32]>;
pub fn capture_dist(dist_dir: &Path, blob: &dyn BlobStore)
    -> Result<(TreeBlob, [u8; 32])>;
```

Tests. Snapshot determinism, build smoke, daemon-restart-during-build recovery.

10.2 P-B2: webapp.* abilities

New files. `src/runtime/agents/webapp_ability.rs`.

Behaviour. Every transition in §4 is implemented, including the long-running build (per TR-INV-1).

Tests. Round-trip every transition; reject `webapp.serve` from `Created`; optimistic concurrency on `webapp.update_config`; build progress receipts loseable.

10.3 P-B3: Hub static + API + actions routing

New files. `src/runtime/hub/static_handler.rs`, `api_handler.rs`, `actions_handler.rs`; extend `router.rs` for `routes_manifest`.

Behaviour.

1. Static handler: TreeBlob path lookup, blob fetch, immutable cache for `/assets/*`.
2. API handler: capture verb, validate against `api_surface`, parse args, invoke `<owner>.<verb>`, map result.
3. Actions handler: capture verb, validate against `actions_surface`, parse args, invoke `<owner>.actions.<verb>`, map result. Defaults to `application/json`; respects `Content-Type: text/eal` returned by the ability.

Tests. Integration: spin up a fixture web_app whose `api`.abilities contains `shop.echo` and whose `actions`.abilities contains `actions.list_for_page`; `POST /api/echo` returns the echoed args; `POST /easynet-for-agents/list_for_page` returns the action descriptors. Unauthorised verbs return 404.

10.4 P-B4: WebSocket proxy + SPA fallback

New files. `src/runtime/hub/ws_handler.rs`.

Behaviour. WS upgrade on `/ws/*`; first text message must be `{ability,args}` JSON; bind to ability stream; forward frames. SPA fallback: serve `dist/index.html` with `no-cache`.

Tests. WS streaming smoke, SPA fallback for unknown paths.

10.5 P-B5: agent_view + meta endpoints + skill seed

New files. `src/runtime/views/webapp/` (3 modules); minor router extensions for the `/__easynet/*` prefix; bundled skill at `src/runtime/skills/easynet-pages-author/SKILL.md`.

Behaviour.

1. `/__easynet/agent` returns the JSON of §7.
2. `/__easynet/health` returns canonical + runtime states.
3. `/__easynet/receipts` returns CanonicalReceipt history.
4. `/__easynet/schema/<verb>` returns the ability's input/output schemas.
5. Skill seeder: at boot, daemon writes the bundled `easynet-pages-author` skill into each agent's workspace under `<workspace>/skills/` and `<workspace>/.agents/skills/`. Idempotent: skips when the skill is already present and up to date.

Tests. agent_view byte-determinism; skill seeder idempotency; replay reconstructs canonical state from receipts.

11 End-to-end worked example

11.1 Setup

```
$ easynet agent add alice --type claude-code
$ easynet-daemon &        # seeds easynet-pages-author skill into
                          # alice's workspace at first contact
$ easynet-hub --bind 127.0.0.1:8787 &
```

11.2 Author the project

Claude Code, having read `easynet-pages-author`'s SKILL.md, asks the daemon to mint state and create the project directory:

```
$ easynet ability invoke alice.webapp.create --args '{
    "slug":"shop",
    "manifest":{ "name":"shop",
                 "description":"Tiny ceramic shop, agent-built.",
                 "visibility":"public" }
}'
{ "state":"Created", "version":1,
```

```
  "project_dir":"~/.easynet/pages-project/shop" }
```

The agent then cds into the project directory and uses its toolchain:

```
$ cd ~/.easynet/pages-project/shop
$ npm create vite@latest . -- --template react-ts
$ # ... write src/App.tsx, register API and actions abilities ...
$ # ... fill in easynet.app.toml as in §3.2 ...
$ easynet ability deploy abilities/shop.list_items     --node alice
$ easynet ability deploy abilities/shop.checkout       --node alice
$ easynet ability deploy abilities/actions.list_for_page --node alice
$ easynet ability deploy abilities/actions.get         --node alice
```

11.3 Build and serve

```
$ easynet ability invoke alice.webapp.build --args '{ "slug":"shop" }' \
    --stream
[build.started   ] command="npm install && npm run build"
[build.progress  ] log_hash=0x12... 8% installed
[build.progress  ] log_hash=0x13... 47% bundled
[build.completed ] artifact_hash=0x4d28ee17... duration=38.2s
{ "state":"Built", "version":2,
  "build_artifact_hash":"0x4d28ee17..." }

$ easynet ability invoke alice.webapp.serve --args '{ "slug":"shop" }'
{ "runtime_state":"Running" }
```

11.4 Browser fetch

```
$ curl -i http://127.0.0.1:8787/default/alice/shop/

HTTP/1.1 200 OK
Content-Type: text/html
ETag: "TSjuFw0pAvjpAQ=="
Cache-Control: max-age=60, must-revalidate
Content-Security-Policy: default-src 'self'; script-src 'self'; ...

<!doctype html>...
```

11.5 API call from the browser

```
POST /default/alice/shop/api/checkout HTTP/1.1
Cookie: easynet_session=sess-abc
{ "item_id":"mug-12oz", "qty":1 }

→ HTTP/1.1 200 OK
{ "order_id":"ord_77a3f", "eta":"2026-04-29" }
```

11.6 Peer agent discovers the actions

A peer agent (bob) lands on the cart page and asks the owner what it can do:

```
$ curl -s -X POST http://127.0.0.1:8787/default/alice/shop/easynet-for-agents/
    list_for_page \
       -d '{"page_path":"/cart"}' | jq
```

```
[
  { "name":"checkout", "kind":"mission",
    "summary":"Buy the items currently in this user's cart.",
    "url":"/easynet-for-agents/get?name=checkout",
    "params_schema":{
      "address":{"type":"string","required":true}
    } },
  { "name":"clear_cart", "kind":"eal",
    "summary":"Empty the cart.",
    "url":"/easynet-for-agents/get?name=clear_cart" }
]

# bob asks the owner to specialise the mission for its concrete
# address input. Owner inlines the literal into a real EAL DAG
# (no $variable — EAL has none; see §6.4).
$ curl -s -X POST "http://127.0.0.1:8787/default/alice/shop/easynet-for-agents/get" \
       -d '{"name":"checkout",
            "params":{"address":"123 Main St"}}'

mission "checkout" {
  let cart = call "shop.cart_contents" on "alice" with {} timeout 5

  let order = call "shop.checkout" on "alice" with {
    items   = cart.output,
    address = "123 Main St"
  } timeout 30
}
```

Two RPCs, exact-match action, address inlined as a literal by the owner. Bob now runs the returned text directly, citing the `actions.get` receipt so the run's first step carries the cross-state-object causal link:

```
$ easynet mission run /tmp/checkout.eal --cite-receipt <actions.get-receipt-id>
```

Each `call` produces a CanonicalReceipt through the existing dispatcher; the first step's invocation envelope carries `causal_context.Scalar` pointing at the `actions.get` receipt, so an auditor walking the chain sees `actions.list_for_page → actions.get → shop.cart_contents → shop.checkout` as one continuous causal thread.

11.7 Audit replay

The web_app's history shows three CanonicalReceipts (create, build, build); each carries a `post_state_hash` and an `owner_signature`. The browser-driven checkout emitted a CanonicalReceipt for an entirely different state_type (an `order` state object owned by alice or alice's order service); that receipt also chains back to alice's signing key. A peer reading both chains can reconstruct, byte-for-byte:

- which build artifact the page was at when checkout happened;
- which API and actions endpoints were declared at that build;
- which mission body `actions.get` returned for the request that ultimately drove the checkout;
- that all of the above was signed by alice.

This is the audit story decisively unavailable on the traditional web.

12 Open questions deferred to RFC-006-B v2

1. SSR with persistent process (`serve.mode = process`).
2. Cross-node hubs, federated reads.
3. Signed asset bundles for SRI.
4. Build sandboxing.
5. Multi-version routing (`webapp.tag` transitions).
6. dev-mode reverse-proxy (`serve.mode = dev-proxy`) for HMR / Vite dev server.

12.1 Coverage check

Main rule	Discharged by	Where
SO-INV-1 (key immutability)	state_key parser refuses key updates	§4
CSH-INV-1 (hash over projection)	schema-driven π, JCS, SHA-256	§4
CSH-INV-2 (content as blob)	TreeBlob + per-file blobs; canonical state holds only hashes	§5
RC-INV-1 (receipt binding)	every webapp receipt carries (state_key, transition_id, attempt_id)	§4
RC-INV-2 (optimistic concurrency)	ETag check on update_config	P-B2
TA-INV-1 (transition_id required)	every `webapp.<verb>@v1` validated	§4
VP-INV-1 (ability_surface from state)	`ability_surface` derived from state_value	§8
TR-INV-1 (long-running terminal)	build emits exactly one terminal CanonicalReceipt	§5
TR-INV-2 (progress not canonical)	`build.progress` are OperationalReceipts	§5
TR-INV-5 (runtime overlay independent)	runtime crash leaves canonical untouched	§4
TR-INV-7 (multi-view first-class)	human (HTML+API+WS) and agent_view both implemented	§7, §8
TR-INV-8 (cross-state-space matrix)	canonical × runtime admissibility enforced	§4
TR-INV-11 (URA-receipt traceability)	`/__easynet/receipts` + state_key index	§11
TR-INV-12 (principal/human-anon)	Hub mints the URI on every request	§7
TR-INV-13 (caller_context)	populated from headers; recorded in operational receipts	§7, §11
WB-INV-1 (artifact-from-blob-only)	Hub never reads project's dist/	§3
WB-INV-2 (api/actions surface enforcement)	Hub refuses unlisted verbs with 404	§3

Closing. The agent-native web's discoverability problem is now solved at the right layer. A peer agent landing on a page asks `actions.list_for_page` and gets exactly the EAL/missions relevant to that page, parameterised by its own intent — not a sitemap to filter, not a manifest to download. Authoring is a learnable skill, not folklore: every agent acquires `easynet-pages-author` on first contact. The project lives at a stable, agent-independent location (`~/.easynet/pages-project/`) so the same codebase reads the same way regardless of which agent operates it. Implementing P-B1 through P-B5 closes Phase 1.

The web evolved from documents to applications; agent-native web evolves from passive views to queryable actions. Pages first, web_apps next, actions are how peer agents make use of either without bulk-downloading anything.